PORTER: A Plan-Committed Router for Multi-Core Distributed Quantum Computers

Fuchao Sheng, Yongjian Xie, Yongming Li, Sanjiang Li *

***Abstract*—A distributed quantum computer executes a circuit across multiple small cores connected by entanglement-distribution links; under the cost model considered here, inter-core communication consumes Einstein–Podolsky–Rosen (EPR) pairs, which the compiler minimises. Choosing an inter-core action is coupled to local routing: a router that scores an action together with the preparation its current layout demands can favour an easy-to-prepare action over one that provides greater routing progress. We present PORTER, a deterministic, plan-committed router that represents each candidate inter-core action as a *plan*: its communication primitive, the gates it can discharge, and its readiness conditions. It ranks plans by progress, uses preparation cost only to break ties, commits to one, and then generates the local SWAPs needed to execute it. It supports TELEDATA, TELEGATE, and same-control TELEGATEPACKET operations, and adds capacity relief, a deterministic initial layout, and a fixed four-horizon teleport-lookahead portfolio. On MQT-Bench suites with 25, 64, and 80 logical qubits, PORTER uses 59.7%, 44.9%, and 68.8% fewer EPR pairs in geometric mean than TELESABRE over the instances both complete, and completes all 24 main-suite instances to TELESABRE's 17. Against DSABRE it is competitive rather than uniformly superior under the respective native protocols, using $0.99\times$ DSABRE's pooled EPR at $1.20\times$ its complete compile time, and it uses $0.69\times$ DSABRE's pairs on a 100–360-qubit QFT and QPEexact scale-up series. The implementation, harness, replay verifier, and raw results are public.**

***Index Terms*—distributed quantum computing, quantum circuit compilation, multi-core quantum architectures, gate teleportation, cat-entangler packets, plan-oriented routing, entanglement cost**

I. INTRODUCTION

Distributed quantum computers (DQCs) execute a circuit across several small quantum cores connected by entanglement-distribution links [1]–[3]. A two-qubit gate whose operands are in different cores requires inter-core communication. Under the cost model considered here, that communication consumes Einstein–Podolsky–Rosen (EPR) pairs; distributing such pairs is much slower and less reliable than operating within a core [4], [5]. We accordingly minimise EPR-pair consumption (Section II-B).

A compiler can use an EPR pair in three ways. TELEDATA moves a logical qubit to another core, potentially making several later gates local [6]. TELEGATE executes one remote controlled gate through a cat-entangler, without moving either operand [7], [8]. A same-control TELEGATEPACKET keeps that resource open for an ordered run of remote gates with a common control, so that the run consumes one pair rather than one pair per gate [9]. The best choice depends on the circuit structure where it is applied: data teleportation has possible future benefit, gate teleportation resolves one gate immediately, and a packet is beneficial only when a suitable run can be found.

This choice is coupled to local routing. Before an inter-core action can execute, its operands may need to be moved to designated communication ports, the ports must be free, and a TELEDATA destination must have space. If a router scores an action together with all of the SWAPs required by the current layout, it can favour an easy-to-prepare action over one that resolves more useful remote work. Moreover, changing the selected remote action during preparation can leave the router with SWAPs that no longer serve its decision.

We present PORTER (*Plan-Oriented Router for Teleportation-Enabled aRchitectures*), a deterministic router that separates action selection from action preparation. PORTER represents each candidate as a *plan*: the chosen inter-core primitive, the gates it can discharge, and the conditions needed to execute it, but not the SWAP sequence that will satisfy those conditions. It first ranks plans by the progress they provide, uses preparation cost only to break ties, commits to one plan, and then generates the local SWAPs needed to make that plan ready. PORTER also includes capacity relief for congested cores and a deterministic initial-layout heuristic. Its operating protocol then routes each circuit at a fixed portfolio of teleport-lookahead horizons.

Problem. Given a circuit, a multi-core architecture, and an initial layout, produce a legal operation stream that implements the circuit while minimising the number of EPR pairs consumed. Section II-B defines the model, objective, and assumptions precisely.

Contributions. This paper makes three contributions.

(i) *A plan-committed distributed router.* PORTER selects a complete inter-core plan before generating plan-specific preparation SWAPs (Section III-D). It combines this stage with a two-tier capacity-relief mechanism and a deterministic initial-layout heuristic (Sections III-D and III-A).

(ii) *A fixed lookahead-horizon portfolio.* Rather than assuming that one teleport-lookahead horizon suits every circuit, PORTER runs the same four horizons and retains the completed route with the lowest EPR count. Since the configured horizon is included and wins ties, this portfolio cannot return a higher EPR count than that configured one-route baseline when the baseline completes (Sec-

F. Sheng, Y. Xie, and Y. Li are with the School of Mathematics and Statistics, Shaanxi Normal University, Xi'an, China; S. Li is with the Centre for Quantum Software and Information, University of Technology Sydney, Australia (e-mail: sanjiang.li@uts.edu.au).

*ORCID: 0000-0002-3332-2546.

tion IV-D1).

(iii) *A controlled empirical evaluation.* We evaluate 48 circuit–architecture instances using native, shared-layout, and cross-layout protocols, and we ablate the principal mechanisms. Every reported PORTER route is checked by an independently implemented replay verifier; the implementation, evaluation harness, and per-circuit results are released.

Results in brief. On MQT-Bench [10] suites with 25, 64, and 80 logical qubits, PORTER reduces geometric-mean EPR consumption relative to TELESABRE [11] by 59.7%, 44.9%, and 68.8%, respectively, on the instances completed by both methods; PORTER completes all 24 main-suite instances whereas TELESABRE completes 17. Compared with DSABRE [12], PORTER is competitive rather than uniformly superior on the main suites: it uses $0.99\times$ DSABRE's pooled EPR count at $1.20\times$ its complete compile time under the respective native protocols. On the 100–360-qubit QFT and QPEexact scale-up series, PORTER uses $0.69\times$ DSABRE's EPR count and $0.52\times$ its compile time, and it obtains the lower EPR count on all six instances.

These are results under the stated protocols. The route budgets differ: PORTER spends four routes against TELESABRE's three seeds and DSABRE's nine.

Organisation. Section II fixes the machine model, the communication primitives, and the problem statement. Section III presents PORTER. Section IV states the research questions, the experimental inputs, and the comparison protocols, including the multi-horizon and refinement protocols that invoke the router repeatedly. Section V reports the results in the order of those questions. Section VI discusses what the evidence does and does not establish and states the open problems. Section VII places PORTER among closely related routers, especially DSABRE, and Section VIII concludes.

The implementation, replay verifier, comparison harness, and raw per-circuit results are available at https://github.com/ebony72/porter. Supplementary material provides benchmark-construction details, external layouts, verifier checks, a worked routing example, per-instance results, parameter settings, complexity analysis, and threats to validity.

II. Model, Primitives, and Problem Formulation

A. Quantum Circuits and Routing

Our input circuits act on n logical qubits and use the gate basis $\{\texttt{id}, \texttt{rz}, \texttt{sx}, \texttt{x}, \texttt{cx}\}$ produced by Qiskit's optimization level 3 for MQT-Bench circuits [10]. Thus, the two-qubit operations considered by the router are controlled-NOT (CX) gates. We represent a circuit as a directed acyclic graph (DAG) with an edge between consecutive gates on each logical qubit. The *front* is the set of unexecuted gates whose predecessors have all executed. Only front gates may execute immediately or be routed toward execution.

The physical qubits are partitioned into K *cores*, each with its own intra-core coupling graph, and selected pairs of cores are joined by an inter-core link whose two endpoints are *communication ports*. The *core graph* has the K cores as its vertices and one edge for each inter-core link; two cores are *adjacent* when the core graph joins them. We assume throughout that the core graph is connected and that the coupling graph of every individual core is connected, so that any logical qubit can in principle be moved to any physical position. In every architecture evaluated here, adjacent cores have exactly one link, and each core uses a distinct port for each neighbour. Hence, once an adjacent pair of cores is chosen, the communication-port pair is fixed, and a plan has no port to choose.

A *layout* ℓ maps each logical qubit to a physical position. A CX gate is *local* when its operands are in the same core and *remote* otherwise. A local CX whose physical operands are not adjacent in the coupling graph requires intra-core SWAPs. A remote CX requires an inter-core communication primitive, possibly preceded by local SWAPs that make the primitive ready. We write $\kappa_\ell(q)$ for the core that holds logical qubit q under ℓ, $d_{\mathrm{L}}(c, p, p')$ for the shortest-path distance between positions p and p' in core c, and $d_{\mathrm{C}}(c, c')$ for the shortest-path distance between cores c and c' in the core graph.

B. Objective, Scope, and Assumptions

Objective. Given a circuit DAG C on n logical qubits, a multi-core architecture $\mathcal{A}$ with K cores, intra-core coupling graphs, inter-core links, designated communication ports, and an initial layout ℓ_0, the compiler must emit a legal operation stream that implements every gate in C. The objective is to minimise the number of consumed EPR pairs. We also record compile time.

Cost model. Each TELEDATA, TELEGATE, and TELEGATEPACKET operation consumes one EPR pair. A packet consumes one pair regardless of the number of remote CX gates that it contains; a multi-hop TELEDATA relocation consumes one pair for each hop. All links have the same EPR-pair cost. This common unit permits comparison with TELESABRE [11] and DSABRE [12].

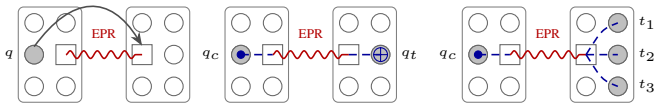
Communication ports are ordinary physical qubits. A port is occupied for the duration of an inter-core operation and is available otherwise; we do not assume dedicated communication memory. The model does not represent link contention, entanglement-generation latency or fidelity, purification, or classical-communication latency. Accordingly, all results concern EPR-pair consumption under this model, rather than scheduled execution time or end-to-end fidelity.

C. Communication Primitives

PORTER uses the three inter-core primitives in Figure 1. Each consumes one EPR pair, but the amount of CX work that can be associated with that pair differs.

State teleportation (TELEDATA). TELEDATA moves a logical qubit from a source communication port to a free destination port in an adjacent core. It consumes one EPR pair. The relocated qubit may then meet subsequent operands in its new core, making their gates local. Its benefit is therefore *latent*: one communication action can reduce the cost of multiple later interactions.

Gate teleportation (TELEGATE). TELEGATE implements one remote CX without moving either logical operand. Write



(a) TELEDATA (b) TELEGATE (c) TELEGATEPACKET

Fig. 1. The three inter-core communication primitives. Each consumes one EPR pair shared by the two communication ports (squares). (a) TELEDATA moves logical qubit q from one core to a free destination port in another core; q must first be brought adjacent to the source port. (b) TELEGATE implements one remote CX through a cat-entangler [8]: the target-side port becomes a proxy for control q_c, and neither logical operand moves. (c) TELEGATEPACKET retains that resource for an ordered same-control run of remote CX gates. Each target is brought next to the target-side port in turn while both ports remain reserved, so the full run uses one EPR pair.

e_A for the communication port in the control's core and e_B for the port at the other end of the link, in the target's core, and let $\alpha|0\rangle+\beta|1\rangle$ be the state of the control q_c. An EPR pair entangles e_A with e_B; a cat-entangler then consumes e_A and leaves the cat state $\alpha|00\rangle+\beta|11\rangle$ on the pair (q_c, e_B), so that e_B carries a copy of the control's computational-basis value. A CX controlled by e_B therefore acts as if it were controlled by q_c; a cat-disentangler restores q_c and releases both ports.

Same-control packet (TELEGATEPACKET). If an ordered sequence of remote CX gates has a common control and all of its targets are in the same core, one cat-entangler/disentangler pair can bracket the sequence. The ports remain reserved while each target is routed adjacent to e_B and its CX is executed. The sequence therefore consumes one EPR pair rather than one pair per CX. This adapts AutoComm's communication-amortisation idea [9] to an online routing loop.

The primitives consequently differ in the respect that governs scoring: how many CX gates one pair is spread over. TELEGATE resolves one CX immediately, TELEGATEPACKET resolves a lookahead-discovered run whose length the compiler must discover, and TELEDATA changes the layout in anticipation of future interactions. Section III-E formalises this comparison in the plan-scoring rule.

III. THE PORTER COMPILER

PORTER is a deterministic heuristic router for a circuit DAG C and an initial layout ℓ_0. At a fixed teleport-lookahead horizon L_{tp}, it repeatedly classifies the DAG front (Section III-B), executes all immediately executable work, routes local CX gates by intra-core SWAPs (Section III-C), and otherwise selects one inter-core *plan*, prepares it, and executes it (Algorithm 1). The plan-first stage separates the value of an inter-core action from the local SWAPs needed to make that action executable. The multi-horizon protocol that invokes this routing pass repeatedly is defined in Section IV-D1.

The implementation additionally bounds the total iterations and consecutive iterations that retire no gate, and detects repeated preparation states. These guards ensure termination of an implementation run (Section III-H).

A. Initial Layout

When no external layout is supplied, PORTER constructs one deterministic layout with at least $f_{\min} = 2$ initially free physical positions in each core. The same value has three distinct uses: it is the initial reserve, the admission threshold for a normal TELEDATA destination, and the free-slot threshold defining a relief reservoir. It is not a persistent lower bound: a teleport admitted at exactly $f_{\min}$ leaves its destination with one free position.

The layout heuristic is temporal locality-greedy. Each CX $g = (q_i, q_j)$ contributes $\gamma^{\text{asap}(g)}$ to an interaction weight $w(q_i, q_j)$, where $\gamma = 0.99$ and $\text{asap}(g)$ is the CX-only DAG layer of g. Logical qubits are visited in first-CX-occurrence order and assigned, when possible, to the core containing their heaviest already assigned neighbour while preserving the reserve. Within a core, more strongly interacting logical qubits receive more central physical positions. This favours early, repeated interactions. Its one weakness is that the reserve is uniform across cores, which suits a grid or a ring but not a core graph with a hub (Section V-D1).

Algorithm 1 PORTER `compile`(C, ℓ_0)

$\ell \leftarrow \ell_0$; initialise the active and front sets of the DAG
while the DAG has active gates and no implementation guard has fired **do**
  $F \leftarrow$ current DAG front; classify its gates
  if a 1Q gate or a directly executable local CX is in F **then**
    execute all such gates; remove them from the DAG; **continue**
  end if
  if a local CX in F needs routing **then**
    apply the best intra-core SWAP under Eq. 1; **continue**
  end if
  generate normal plans $\mathcal{P}$ and blocked plans $\mathcal{B}$
  score all plans and choose the first normal plan in key order
  if no normal plan is available **then**
    choose a tail-relief plan seeded by $\mathcal{B}$
  else if $\mathcal{B} \neq \emptyset$ and its best member exceeds the selected normal plan by more than m_{assist} **then**
    replace the selection by an opportunistic-relief plan
  end if
  if no plan was selected **then**
    report capacity deadlock; abort
  end if
  prepare the selected plan by local SWAPs, then execute it
end while
return emitted operations, final layout, and statistics

B. Front Classification

Under layout ℓ, every front gate is one of the following:

- a 1Q gate, which is directly executable;
- a direct-local CX, whose operands are adjacent in one core;
- a local CX needing SWAPs, whose operands share a core but are not adjacent;

- an adjacent-remote CX, whose operand cores are neighbours; or
- a non-adjacent-remote CX.

PORTER drains directly executable gates first. It next routes local CX gates, and enters the remote stage only when every front CX is remote. This priority keeps ordinary intra-core routing separate from inter-core decisions.

C. Intra-Core SWAP Scoring

For the local-CX set F_{loc}, candidate SWAPs are coupling-graph edges incident to positions touched by F_{loc}. A candidate a is evaluated under its post-SWAP layout ℓ_a by the SABRE-style objective

$$J(a) = \bar{d}_{\mathcal{F}}(a) + \lambda_{\text{loc}} \bar{d}_{\mathcal{E}}(a), \qquad \lambda_{\text{loc}} = 0.8, \tag{1}$$

where the two terms are mean intra-core distances over F_{loc} and over up to $N_{\text{loc}} = 20$ additional local CX gates from the next $L_{\text{loc}} = 12$ DAG layers. The router applies the candidate with the smallest J, breaking ties by front-layer distance and then deterministically.

D. Plans and the Plan-First Remote Stage

A *plan* is one candidate inter-core action across a single link. It records its primitive, its source and destination cores, the link and ports it uses, the gates it discharges, and the front CX it was generated from — for a relief plan, the CX behind the blocked plan it serves rather than one the action itself advances. It deliberately does *not* record a preparation-SWAP sequence. Thus plans answer "what remote action should be taken next?"; preparation answers "how do we make that action executable from the present layout?" (Section III-F).

The primitive is one of $\{\text{TELEDATA}, \text{TELEGATE}, \text{TELEGATEPACKET}\}$. A TELEDATA plan moves one logical qubit by one core-graph hop and immediately discharges no CX. A TELEGATE plan discharges one adjacent-remote CX. A TELEGATEPACKET plan discharges an ordered, same-control sequence of adjacent-remote CX gates (Section III-G). For the gate-teleportation family, the plan's source and destination denote the control-side and target-side cores; for TELEDATA they denote the sending and receiving cores.

Three kinds of plan. Every plan the router handles is *normal*, *blocked* or *relief*. The three share the record above, the score of Section III-E and the readiness predicate of Section III-F; they differ in how they are generated and in what the router is permitted to do with them.

Normal plans $\mathcal{P}$ are the actions that could fire once prepared, generated from the remote front. For each adjacent-remote front CX, $\mathcal{P}$ contains a TELEGATE plan, a packet plan when a packet of at least two gates is available, and each direction of TELEDATA whose receiving core has at least $f_{\min}$ free positions. For a non-adjacent-remote CX it contains only one-hop TELEDATA moves that strictly reduce the distance between operand cores and meet the same capacity test. PORTER commits to the best of them.

Blocked plans $\mathcal{B}$ are the TELEDATA moves that would otherwise be generated but whose destination fails that capacity test. They are never executed, since committing one would drive a core below the floor. They are nevertheless generated and scored, because each of them records a saturated destination core together with the score of the move that core is refusing. Read as a set, $\mathcal{B}$ therefore reports which cores are congested and how much progress is being forgone at each, and this is the only information the two relief rules below consult.

Relief plans are generated on demand, to create space for a blocked one. A relief plan is always generated *from* a particular blocked plan, and c denotes that blocked plan's own destination core — the core whose shortage stopped it. A breadth-first search from c finds a nearest *reservoir*, a core with at least $f_{\min}$ free positions; the relief plan is the TELEDATA moving one logical qubit from the penultimate core of that path into the reservoir. Only that final hop is committed, and after it executes the router regenerates every plan, so a multi-hop route propagates free capacity one hop at a time rather than reserving an atomic relay (Section VII). A relief plan is always newly generated for the purpose; the blocked plan that motivated it is never itself committed.

When each fires. Normally the best member of $\mathcal{P}$ is committed. Two rules override that, and they consult $\mathcal{B}$ differently. *Tail relief* fires reactively, when $\mathcal{P}$ is empty and the front is starved: a reservoir search is run from the destination core of *every* blocked plan, and the relief plans so generated compete with one another, shortest search path first and then on the ordinary selection key. *Opportunistic relief* fires pre-emptively, while a normal plan is still available, when the *best-scoring* blocked plan out-scores that normal plan by more than $m_{\text{assist}} = 5.0$; only that one blocked plan's destination core seeds the search, the judgement being that clearing the congestion it names is worth more than taking the best currently legal action. Both tiers commit a single hop, and scoring relief on the same key as ordinary work is what lets the opportunistic tier compete with it.

E. Plan Scoring and the Selection Key

Every candidate receives the lexicographic key

$$\text{key}(\mathsf{p}) = \big(-\text{score}(\mathsf{p}),\ \text{prep}(\mathsf{p}),\ -f_{\text{dst}}(\mathsf{p})\big), \tag{2}$$

ordered from smallest to largest, with a stable identifier as the final tie break. Here f_{dst} is the free capacity of a TELEDATA destination and is defined to be zero for the TELEGATE family. Hence the primary comparison is always score; readiness cost matters only when the score ties, and capacity is a final tie break. In particular, it also implements a TELEDATA-first preference when a teleport and a gate plan tie on score, because an admissible teleport has $f_{\text{dst}} \geq f_{\min}$.

score ranks plans; it is not a common physical unit: gate teleportation counts CX gates discharged immediately, whereas a data teleport is valued by current and predicted reductions in core distance. Specifically, a TELEGATE has score $= 1$, and a packet has score equal to the number of CX gates in its collected sequence.

For a TELEDATA plan moving q from c_{src} to c_{dst}, let κ and κ' map logical qubits to cores before and after that one-hop move. The progress of a CX $g = (q_i, q_j)$ is

$$\Delta(g) = d_{\mathrm{C}}\big(\kappa(q_i), \kappa(q_j)\big) - d_{\mathrm{C}}\big(\kappa'(q_i), \kappa'(q_j)\big). \tag{3}$$

Thus $\Delta(g) \in \{-1, 0, 1\}$. Forward simulation under κ' forms three disjoint sets of CX gates incident on q: $D(\mathsf{p})$ contains the seed front CX; $L(\mathsf{p})$ contains CXs exposed while locally executable gates are repeatedly retired; and $A_k(\mathsf{p})$ contains gates reached after that local closure in simulated layer k, for $1 \le k \le L_{\mathrm{tp}}$. The first two describe immediate/local-closure progress; the third is lookahead. With $w_k = \max(w_{\min}, w_0\gamma^{k-1})$, $w_0 = 0.9$, $\gamma = 0.9$, and $w_{\min} = 0.5$,

$$\begin{aligned} \mathrm{direct}(\mathsf{p}) &= \sum_{g \in D(\mathsf{p}) \cup L(\mathsf{p})} \Delta(g), \\ \mathrm{lookahead}(\mathsf{p}) &= \sum_{k=1}^{L_{\mathrm{tp}}} \sum_{g \in A_k(\mathsf{p})} w_k \Delta(g), \\ \mathrm{score}(\mathsf{p}) &= \mathrm{direct}(\mathsf{p}) + \mathrm{lookahead}(\mathsf{p}). \end{aligned} \tag{4}$$

The best member of $\mathcal{P}$ under key (2) is committed, unless one of the two relief rules of Section III-D overrides that choice.

F. Commitment and Preparation

A plan is *ready* in layout ℓ, written $\mathrm{ready}(\mathsf{p}, \ell)$, when the required logical endpoint(s) are adjacent to their designated communication ports, the required ports are available, and, for TELEDATA, the destination satisfies the capacity rule. The readiness heuristic $\mathrm{prep}(\mathsf{p}, \ell)$ is the sum of the intra-core distances from required endpoints (or the destination free slot) to their required ports. It is zero exactly for a ready plan.

Because plans are scored before any plan-specific preparation SWAP exists, the choice is made on what an action is worth rather than on what it would cost to set up. PORTER commits to the selected plan, performs only local SWAPs intended to reduce its readiness cost, and executes the plan once it is ready. It does not reopen the remote choice during that preparation. Commitment therefore prevents local moves made for one remote action from being reinterpreted as preparation for a different action.

For an unmet endpoint requirement, preparation considers incident intra-core SWAPs. For an occupied TELEDATA destination port, it considers SWAPs that move a nearest free position toward that port without swapping through the port prematurely. Candidates are ranked by post-SWAP prep, and at most one move per plan side is applied in a preparation round.

Under the stated connectivity and capacity assumptions, a legal preparation sequence exists for a committed plan. Every core retains a free position, a teleport entering a destination only when it has at least $f_{\min} = 2$ and therefore leaving one, so on a connected core that free position can be walked to an occupied port to clear it, and the endpoint then walked to a neighbour of that port along a path meeting no edge incident to it — at most a core diameter of local SWAPs apiece. The two sides lie in different cores and a local SWAP moves no qubit between cores, so neither side can undo the other and the capacity rule survives preparation.

G. Same-Control Packet Collection

A packet is discovered by forward simulation because CX gates sharing a control are generally chained in the DAG and therefore do not appear in the front together. From an adjacent-remote seed $g = (q_c, q_t)$ whose target lies in core c_t, the collector simulates up to 120 layers. It absorbs up to eight front CX gates with control q_c and target in c_t, together with executable 1Q gates already in c_t, and stops at the first unproductive layer or the size cap. A packet is generated only if at least two CX gates are collected.

Packet readiness means readiness to establish the cat resource, not readiness of every later target. Once committed, the router emits a single cat-entangler; then, for each recorded target in turn, it routes that target beside the target-side port without disturbing either reserved port and emits the target's CX against that port; and it finally emits one cat-disentangler. The recorded order follows the DAG, so the packet plan may safely count all of its members as discharged work.

H. Progress and the Failure Model

The capacity discipline prevents routing from creating a completely full core: every normal or relief TELEDATA is admitted only when its destination has at least two free positions, and therefore leaves at least one. A supplied layout with a full core is first legalised by an eviction if another core can receive it; otherwise the input is infeasible.

This invariant does not prove universal successful routing. If normal plans are absent but blocked plans identify a reachable reservoir, tail relief creates a new candidate. If no such candidate exists, PORTER reports a capacity deadlock. Independently, the implementation bounds every loop — repeated preparation states, an iteration budget and a no-progress limit — safeguards on the greedy realisation of the policy rather than conditions the argument of Section III-D needs. Thus every emitted route is checked for legality, and every implementation run terminates, but successful completion on every feasible input is not claimed.

IV. EXPERIMENTAL METHODOLOGY

This section specifies the evaluation questions, experimental inputs, and comparison protocols. A *route* is one invocation of the router on one circuit from one initial layout. A *protocol* specifies how layouts are obtained, how many routes are run, and how the reported route is selected. Reported compile time is the cost of the complete protocol, including layout generation and all unsuccessful candidate routes.

A. Research Questions

The experiments address four questions.

RQ1 Under each compiler's native protocol, how do EPR-pair consumption, completion, and complete compile time compare?

RQ2 When the initial layout is controlled, how much of the difference between PORTER and DSABRE is due to routing rather than layout generation?

RQ3 What are the individual effects of capacity relief, the communication primitives, initial layout, and horizon selection?

RQ4 Do these conclusions persist as circuit width, core count, core shape, and core-graph topology change?

Section V follows this order. Each results subsection states what it controls and what it measures before interpreting the outcome.

B. Experimental Setup

Implementation and checking. PORTER is implemented in Python and released with the comparison harness. It is deterministic: ties follow a fixed key order and any residual tie uses a constant-initialised generator. Thus, a fixed circuit, machine, layout, and horizon produce a byte-identical operation stream. A separately implemented replay verifier checks every reported PORTER stream for gate coverage, dependency order, and physical legality, and the verifier is itself tested by fault injection.

Architectures. We use the two grid-of-grids devices evaluated by DSABRE [12] and introduced in TELESABRE: architecture A, a 2×2 array of 3×3 cores (36 physical qubits), and architecture H, a 2×3 array of 4×4 cores (96 physical qubits). RQ4 also uses DSABRE's 5×5-core scale-up series and two networks of IBM 27-qubit heavy-hex tiles. Device descriptions are read from the baselines' own files so that all compilers use identical physical-qubit identifiers.

Benchmarks. The main inputs are the MQT-Bench files used in the DSABRE evaluation [10], verified by exact CX-count match. Architecture A uses six 25-qubit families (AE, GHZ, Graphstate, QFT, QNN, and Random). Architecture H uses nine 64-qubit families, adding QPEexact, QAOA, and Multiplier. We also construct an 80-qubit suite of the same nine families; its AE and QNN instances are reconstructed because the stock generator does not produce them.

Baselines and execution. We compare with TELESABRE [11] and DSABRE [12]. DSABRE uses its shipped safe mode and layout rule. Its recovery budget is derived from the architecture as in its drivers, and its pre-routing layout-repair EPR count is excluded for both compilers because it changes the starting layout rather than routing the circuit. The stock TELESABRE binary aborts reproducibly in its initial-layout pass whenever no core holds more free positions than its configured margin, which is every 25- and 80-qubit instance and no 64-qubit one. We therefore use and release a minimal patch that assigns the logical qubits the stock code leaves unplaced to the least-loaded core and changes nothing else, so results agree with the stock binary wherever it completes. Each compiler invocation has a 900-s wall-clock limit; an invocation that exceeds it is recorded as incomplete.

C. Layouts and Comparison Protocols

We use native, shared-layout, and cross-layout comparisons. These controls separate the quality of an initial layout from the quality of the routing loop, but they do not make all resource budgets identical.

Native protocols (RQ1 and RQ4). TELESABRE uses its Hungarian layout heuristic at seeds 0–2 and reports the best EPR result. DSABRE uses Qiskit `SabreLayout` at seeds 0–2, followed for each candidate by its forward–backward–forward routing schedule, and reports the best result. PORTER builds one deterministic temporal-locality-greedy layout (Section III-A), routes it at four horizons, and retains the lowest-EPR route. PORTER+R additionally performs backward refinement from that selected route.

For DSABRE, `SabreLayout` operates on its shipped corners-removed coupling map. The removed low-degree corner positions begin empty and provide capacity for arrivals and evictions.

Shared layouts (RQ2). Both DSABRE and PORTER route the same externally supplied layout, the `pytket-dqc` hypergraph partition. This controls the initial mapping but not search effort: DSABRE uses three routing passes, PORTER four horizons, and PORTER+R eight routes. It therefore isolates layout source rather than ranking routing loops at a matched budget.

Cross layouts (RQ2). Each compiler also routes the other compiler's native layout(s). In particular, PORTER routes all three of DSABRE's candidate layouts and selects the lowest-EPR outcome. This test shows whether a native-protocol result is sensitive to the layout source.

D. Operating Protocols

Section III defines one routing pass. The following protocols repeat that pass; they do not change the router itself.

1) PORTER*: fixed horizon portfolio:* The teleport-lookahead horizon L_{tp} determines how many DAG layers are examined when scoring a TELEDATA candidate. An offline sweep over $\{4, 8, 16, 32, 48, 64, 128, 256\}$ found no single best value: the per-instance optimum spans that whole range, and within the portfolio below 80-qubit QNN selects 16 where all six 100–360-qubit scale-up instances select 128. We therefore apply the same fixed portfolio to every circuit,

$$L_{\mathrm{tp}} \in \{4, 8, 16, 128\},$$

route once at each horizon, and retain the completed route with the fewest EPR pairs.

The configured default, $L_{\mathrm{tp}} = 16$, is evaluated first and wins ties, so whenever it completes the portfolio cannot report a higher EPR count than that one-route configuration. This follows from including the default among the candidates; it implies nothing about horizons outside the set.

Its cost is compile time: four routing invocations instead of one, and long lookahead is expensive on dense circuits. Reported protocol times include every horizon probe, successful or not (Section IV-E).

2) PORTER+*R: backward refinement:* PORTER+R applies SABRE-style bidirectional refinement [13] to complete routes. Let $R(C, \ell)$ denote routing circuit C from layout ℓ. Pass 1 routes C from ℓ_0. Pass 2 routes the reversed circuit from the final layout of pass 1 and retains only its final layout. Pass 3

TABLE I
CONFIGURATIONS USED IN SECTION V: LAYOUT SOURCE, LAYOUT COUNT $|\mathcal{L}|$, ROUTES SPENT PER LAYOUT P, TOTAL ROUTES, AND WHAT IS SELECTED. A ROUTE IS ONE ROUTER INVOCATION. ON THE PORTER ROWS, P COUNTS THE FOUR CANDIDATE HORIZONS; ON THE +R ROWS IT COUNTS THE FIVE REFINEMENT PASSES, OF WHICH THE FIRST IS ITSELF THE FOUR-HORIZON PORTFOLIO, GIVING $4 + 4 = 8$ ROUTES.

Configuration	Layout source	$\lvert\mathcal{L}\rvert$	P	Routes	Selection rule
Native protocol (RQ1, RQ4)					
TELESABRE	Hungarian, seeds 0–2	3	1	3	lowest EPR
DSABRE	SabreLayout, seeds 0–2	3	3	9	lowest EPR
PORTER	temporal locality-greedy	1	4	4	lowest EPR
PORTER+R	temporal locality-greedy	1	5	8	best forward route
Shared layout (RQ2)					
DSABRE shared	one external layout	1	3	3	best pass
PORTER shared	the identical layout	1	4	4	lowest EPR
PORTER+R shared	the identical layout	1	5	8	best forward route
Cross layout (RQ2)					
DSABRE on PORTER	PORTER layout	1	3	3	best pass
PORTER on DSABRE	three DSABRE layouts	3	4	12	lowest EPR
Ablation (RQ3)					
PORTER ablation	temporal locality-greedy	1	1	1	—

routes C from that layout. We use five passes by repeating this forward–backward cycle and report the best forward route.

The first pass is the four-horizon PORTER portfolio. Its selected horizon is retained for the remaining four passes, rather than being reselected at each pass. Thus PORTER+R executes eight routes in total, of which the first four are exactly the routes PORTER runs; since the best forward route is the one reported, PORTER+R can never report more EPR pairs than PORTER.

E. Budgets and Metrics

Table I makes the search budget explicit. "Routes" counts all routing invocations, including horizon probes. Thus route count is a transparent effort descriptor, not a claim that routes have equal runtime.

The primary outcome is EPR pairs consumed. TELESABRE counts TELEDATA and TELEGATE operations; DSABRE counts TELEDATA operations; and PORTER counts TELEDATA, TELEGATE, and TELEGATEPACKET operations, one pair per packet. Under the cost model of Section II-B, these are the same physical resource. We also report wall-clock compile time.

Aggregate EPR and compile-time results are unweighted geometric means over circuits. For a comparison of methods a and b, the reported ratio is b/a and the percentage change $100(b/a - 1)$, both computed from the aggregate means. A ratio below one, equivalently a negative percentage, means b uses fewer pairs: PORTER's pooled 169.4 against DSABRE's 171.9 is reported as $0.99\times$, a 1.5% reduction.

Failures are not imputed: comparisons exclude unsuccessful invocations rather than assigning them a penalty value. Each paired mean therefore covers only circuits completed by every column in it, and reports its completion count. Since that can flatter a baseline by dropping the instances it could not route, completion counts are co-primary outcomes.

Compile times are complete protocol wall-clock times measured once per configuration on an otherwise idle Apple M2 system (macOS 26.5, CPython 3.13), without warm-up. They should be interpreted as indicative protocol-level ratios, not as portable absolute timings.

V. RESULTS

This section answers RQ1–RQ4 (Section IV-A). Table I defines the protocol names, routing-invocation counts, and complete compile-time accounting. Lower EPR consumption is better throughout.

A. RQ1: Native-Protocol Comparison

Question: How do the compilers compare when each uses its native layout and routing protocol? *Control:* none; this is an end-to-end comparison. *Outcomes:* geometric-mean EPR consumption over the stated completed instances, completion count, and complete protocol time.

Comparison with TELESABRE. On the common completed subsets in block (a), PORTER reduces geometric-mean EPR consumption relative to TELESABRE by 59.7%, 44.9%, and 68.8% at 25, 64, and 80 qubits, respectively. Every circuit completed by TELESABRE improves, with per-instance reductions from 23.2% to 76.9%. PORTER+R changes the suite-level reductions to 63.5%, 46.9%, and 70.3%. TELESABRE completes 17 of 24 main-suite instances; it does not complete 64q Random or six 80-qubit instances. PORTER and DSABRE complete all 24.

Comparison with DSABRE: native-protocol outcome. Both methods complete all 24 main-suite instances, so block (b) gives the primary comparison. PORTER uses $0.94\times$ DSABRE's geometric-mean EPR count at 25 qubits and $0.93\times$ at 80 qubits, but $1.07\times$ at 64 qubits. Over all 24 instances, PORTER uses 169.4 EPR pairs in geometric mean versus DSABRE's 171.9, or $0.99\times$ the DSABRE count. PORTER+R reduces the pooled result to 156.7 ($0.91\times$ DSABRE), while increasing complete compile time from 1.68 to 3.00 seconds per instance.

TABLE II

NATIVE-PROTOCOL RESULTS. BLOCK (A) INCLUDES ONLY CIRCUITS COMPLETED BY ALL FOUR REPORTED CONFIGURATIONS AND IS THEREFORE THE VALID COMPARISON WITH TELESABRE. BLOCK (B) INCLUDES EVERY CIRCUIT IN EACH SUITE AND OMITS TELESABRE. t IS COMPLETE PROTOCOL TIME (GEOMETRIC MEAN, SECONDS). "W/L/T" COMPARES THE NAMED PORTER PROTOCOL WITH DSABRE OVER ALL 24 INSTANCES. BOLD MARKS THE LOWEST EPR COUNT AMONG THE FOUR REPORTED CONFIGURATIONS IN EACH ROW. THE TWO RIGHTMOST COLUMNS GIVE THE PER-INSTANCE MINIMUM OF DSABRE AND THE NAMED PORTER PROTOCOL, TAKEN AFTER BOTH HAVE RUN; THEY MEASURE HOW FAR APART THE TWO COMPILERS' OUTCOMES FALL AND ARE LEFT UNBOLD BECAUSE THEY ARE NOT A SINGLE COMPILER.

	TELESABRE		DSABRE		PORTER		PORTER+R		oracle minimum with	
Suite	EPR	t	EPR	t	EPR	t	EPR	t	PORTER	+R
(a) over the circuits *all four* complete										
25q (6)	106.6	0.51	45.8	0.23	43.0	0.21	**38.9**	0.36	41.9	38.9
64q (8)	284.5	11.89	**144.6**	1.82	156.8	2.24	151.1	3.93	140.5	139.6
80q (3)	121.0	5.11	44.9	3.59	37.8	5.00	**35.9**	9.35	33.8	33.8
(b) over *every* circuit in the suite; TELESABRE omitted										
25q (6)	—	—	45.8	0.23	43.0	0.21	**38.9**	0.36	41.9	38.9
64q (9)	—	—	**173.1**	1.82	186.1	2.24	179.6	3.93	168.8	167.4
80q (9)	—	—	412.1	3.59	384.7	5.00	**346.1**	9.35	344.7	333.4
all (24)	—	—	171.9	1.40	169.4	1.68	**156.7**	3.00	155.7	150.5
done	17 / 24		24 / 24		24 / 24		24 / 24		—	
W/L/T	—		—		11 / 10 / 3		14 / 8 / 2		—	

TABLE III

GEOMETRIC-MEAN EPR BY INITIAL-LAYOUT SOURCE AND ROUTING PROTOCOL. BLOCK (A) INCLUDES ALL 24 INSTANCES. BLOCK (B) EXCLUDES GHZ AND GRAPHSTATE, WHOSE VERY SMALL EPR COUNTS DOMINATE AN UNWEIGHTED GEOMETRIC MEAN. WITHIN EACH BLOCK, BOLD MARKS THE LOWEST ENTRY DOWN EACH ROUTING-PROTOCOL COLUMN; IT DOES NOT COMPARE ROUTERS ACROSS A ROW.

Initial layout	DSABRE	PORTER	PORTER+R
(a) all twenty-four instances			
DSABRE's `SabreLayout`	171.9	170.3	156.3
PORTER's temporal locality-greedy	171.2	169.4	156.7
`pytket-dqc`'s partition	**164.1**	**167.2**	**153.8**
(b) the eighteen excluding GHZ and Graphstate			
DSABRE's `SabreLayout`	**452.5**	448.7	426.2
PORTER's temporal locality-greedy	473.9	**445.2**	**422.7**
`pytket-dqc`'s partition	501.1	509.9	461.8

Per-instance variation. PORTER obtains the lower EPR count on 11 instances, DSABRE on 10, and the methods tie on 3. PORTER+R has 14 wins, 8 losses, and 2 ties against DSABRE. The largest PORTER+R saving is on 80q Multiplier (2,130 versus 4,841); its largest loss is on 80q Random (10,848 versus 10,551).

Complementarity. The two rightmost columns measure how differently the two compilers behave. Selecting the lower EPR count for each instance, after both have run, produces geometric means of 155.7 with PORTER and 150.5 with PORTER+R — below either compiler's own pooled mean, because each takes some instances the other loses.

B. RQ2: Routing With Layout Controlled

Question: Given the same initial layout, do the routing loops differ materially in EPR cost? *Control:* layout source. *Outcome:* geometric-mean EPR over all 24 instances. The schedules are not budget-matched: DSABRE uses three passes per layout, PORTER four horizon routes, and PORTER+R eight routes.

TELESABRE appears in neither role. It cannot supply a routing column, because its released implementation always constructs its own initial layout and exposes no interface through which a layout computed elsewhere could be supplied. It is also a poor layout row, since it emits a layout only on the 17 instances it completes. For a fixed layout, DSABRE and PORTER are close: across the rows of Table III, their difference is at most 1.9% in block (a) and at most 6.1% in block (b). By contrast, changing the layout source produces a 9.2–14.5% spread across protocols on the 18 non-trivial instances. The layout source therefore produces a larger EPR spread than the choice between the two routing loops.

The `pytket-dqc` partition is best in every column over all 24 instances, but worst in every column after GHZ and Graphstate are excluded. Its advantage is therefore concentrated in those simple families. It is the worse start on 14 of 24 instances for DSABRE and 16 of 24 for PORTER. The partition optimises e-bits on a network whose communication assumptions are more permissive than those used here [12]. Moreover, three KaHyPar seeds cost 0.6–627 seconds, compared with 0.3–9.0 ms for PORTER's layout construction, and the partitioner does not finish on the scale-up inputs. We therefore retain PORTER's layout as the default and treat this partition as an opt-in alternative.

Backward refinement improves every layout row: it reduces EPR by 8.2% from DSABRE's layout, 8.0% from the partition, and 7.5% from PORTER's layout.

C. RQ3: Mechanism Ablation

Question: Which mechanisms affect EPR consumption or completion? *Control:* one forward route from the same layout at the configured horizon; all other settings are fixed. *Outcomes:* EPR and completion. The selected multi-horizon result in Table II is not used here.

Completion. Only the full configuration routes all nine 80-qubit circuits. Disabling both relief tiers leaves four, restricting

TABLE IV
80-QUBIT ABLATION. "NO PK" DISABLES TELEGATEPACKET; "NO REL." DISABLES BOTH RELIEF TIERS; "TELEDATA" REMOVES THE TELEGATE FAMILY AND ALSO CHANGES RELIEF ELIGIBILITY. "PK"/"TG" ARE THE FULL CONFIGURATION'S PACKET AND SINGLE-TELEGATE COUNTS. † DENOTES NON-COMPLETION. GEOMETRIC MEANS ARE REPORTED ONLY OVER ROWS COMPLETED BY BOTH VARIANTS IN THE STATED COMPARISON.

Circuit	Full	No pk	No rel.	TELEDATA	pk	tg
ae	331	367	†	342	54	1
ghz	7	7	7	7	0	0
graphstate	28	28	28	28	0	0
qft	318	337	†	†	78	0
qnn	1072	1099	†	1132	113	2
random	11245	†	†	†	537	3792
qpeexact	435	443	†	**347**	124	0
qaoa	1202	1146	1505	†	360	21
multiplier	3897	3584	3055	†	208	1739
gmean, n=8 of 9	**286.2**	288.9				
gmean, n=4 of 9	174.1		**173.3**			
gmean, n=5 of 9	124.8			**121.4**		

the vocabulary to TELEDATA leaves five, and disabling packets leaves eight (Table IV). Completion, not EPR count, is where three of the four mechanisms are decisive.

Packets. On the eight circuits completed with and without packets, the geometric-mean EPR count is unchanged within 0.9%, and the direction varies by circuit: packets reduce cost on AE, QFT, QNN, and QPEexact and raise it on QAOA and Multiplier. Packets matter more at smaller widths, saving 3.0% at 25 qubits and 7.4% at 64 qubits.

Gate teleportation. Deleting the TELEGATE family alone deadlocks seven of the nine circuits. A TELEGATE needs no free data position in the target core, because the cat-state copy occupies the target-side communication port; and while a TELEGATE plan exists for an adjacent remote CX, the last-resort relief tier never receives an adjacent blocked plan. The TELEDATA column therefore also re-admits those blocked plans to the relief tier, a change that is inert whenever a TELEGATE is available. It measures a teleport-only vocabulary rather than the removal of one mechanism. Five circuits complete, using 121.4 EPR pairs in geometric mean against 124.8 for the full configuration.

Capacity relief. Disabling both relief tiers prevents completion on five of the nine 80-qubit circuits. Relief earns its place on completion rather than cost: on the four circuits completed either way the full configuration uses 174.1 EPR pairs against 173.3.

D. RQ4: Robustness to Scale and Topology

Scale-up question: Do the native-protocol results persist as circuit width and core count increase together? *Control:* DSABRE's published scalability architectures and circuit files. *Outcomes:* EPR and complete compile time.

The six-instance scale-up series of Table V contains QFT and QPEexact only. It therefore shows that the reported native-protocol outcome persists for these two families as width and core count grow; it does not establish a family-independent

TABLE V
SCALE-UP RESULTS UNDER NATIVE PROTOCOLS. THE 5×5-CORE ARCHITECTURES GROW FROM 6 TO 20 CORES, READ FROM DSABRE'S OWN DEVICE FILES; OUR DSABRE COLUMN REPRODUCES ITS PUBLISHED SERIES [12]. t IS COMPLETE PROTOCOL TIME. PORTER SELECTS $L_{\mathrm{tp}} = 128$ FOR EVERY INSTANCE.

		architecture			DSABRE		PORTER	
Circuit	CX	n	N_{phys}	K	EPR	t	EPR	t
QFT	3,420	100	150	6	200	3.6	**198**	3.0
QPEexact	3,579	100	150	6	246	4.3	**200**	2.4
QFT	7,220	200	300	12	765	13.9	**564**	9.3
QPEexact	7,579	200	300	12	896	13.9	**528**	9.0
QFT	13,300	360	500	20	1890	53.9	**1074**	18.6
QPEexact	13,979	360	500	20	1938	83.1	**1093**	22.5
gmean (6)					705.7	15.4	**490.0**	**8.0**

TABLE VI
FOUR IBM 27-QUBIT HEAVY-HEX CORES (108 PHYSICAL QUBITS), CONNECTED AS A RING OR A STAR. THE RIGHTMOST COLUMN GIVES PORTER ON THE THREE DSABRE SABRELAYOUT CANDIDATES. BOTH PROTOCOLS SELECT THEIR BEST RESULT, SO THIS CONTROLS CANDIDATE-LAYOUT SOURCE BUT NOT ROUTE COUNT.

		PORTER	
Core graph	DSABRE	own layout	DSABRE layouts
ring (9)	85.2	86.1	**82.9**
star (9)	101.1	113.3	**97.2**

ranking for large circuits. On all six instances, PORTER uses fewer EPR pairs than DSABRE: 0.90× its geometric-mean EPR count at 100 qubits, 0.66× at 200 qubits, 0.57× at 360 qubits, and 0.69× overall. Its complete protocol time is 8.0 versus 15.4 seconds, or 0.52× the DSABRE time.

On these six instances, the fixed four-horizon portfolio accounts for the reported EPR improvement. Routing only at the configured $L_{\mathrm{tp}} = 16$ yields a geometric mean of 1,020.7 pairs (1.45× DSABRE), whereas the selected portfolio result is 490.0 pairs (0.69× DSABRE); every instance selects $L_{\mathrm{tp}} = 128$.

1) Heavy-hex cores and core graphs: Question: Does the result depend on square cores connected in a grid? *Control:* the nine 64-qubit circuits and 108 physical qubits; core shape and core graph vary. *Outcome:* geometric-mean EPR.

On the ring, the native protocols are within 1.1% (86.1 versus 85.2). On the star, PORTER's native layout is worse (113.3 versus 101.1), but PORTER reaches 97.2 when supplied with DSABRE's candidate layouts (Table VI). This points to a layout–topology interaction: a uniform reserve treats cores similarly, whereas the star hub carries disproportionate traffic.

E. EPR/Time Trade-off

Table VII summarises the EPR/time trade-off among the evaluated protocols. PORTER+R runs PORTER's four routes and then four more, and reports the best forward route, so both columns are monotone by construction. The table therefore prices refinement rather than ranking two independent protocols. Relative to DSABRE, PORTER uses 0.99× the pooled

TABLE VII
PROTOCOL TRADE-OFF OVER ALL 24 MAIN-SUITE INSTANCES. TIMES ARE COMPLETE PROTOCOL COSTS (GEOMETRIC MEAN PER INSTANCE).

Protocol	routes	EPR	vs. DSABRE	time (s)	vs. DSABRE
DSABRE	9	171.9	1.00	1.40	1.00
PORTER	4	169.4	0.99	1.68	1.20
PORTER+R	8	156.7	0.91	3.00	2.15

geometric-mean EPR count at $1.20\times$ the complete protocol time. PORTER+R reduces EPR to $0.91\times$ DSABRE's count at $2.15\times$ its time. These are complete protocol costs: they include layout construction and every candidate route.

VI. DISCUSSION AND FUTURE WORK

The evidence supports a qualified native-protocol conclusion. Across the 24 main-suite instances, PORTER and DSABRE have nearly equal pooled geometric-mean EPR counts (169.4 and 171.9, respectively), whereas PORTER+R reduces PORTER's count to 156.7 at a higher complete compile-time cost (Table II). Neither router dominates per instance: PORTER records 11 wins, 10 losses, and 3 ties against DSABRE, while PORTER+R records 14 wins, 8 losses, and 2 ties. Thus, under the reported native protocols, PORTER is competitive with DSABRE rather than uniformly superior.

The two routers are complementary. They obtain their lower EPR counts on different instances. Selecting the lower count for each instance gives 155.7 with PORTER and 150.5 with PORTER+R, which are 9.4% and 12.5% below DSABRE's pooled geometric-mean EPR count, and 8.2% and 10.3% below it once GHZ and Graphstate are excluded. The margin is a property of the pair rather than of either compiler, and it indicates that the two designs are not interchangeable.

The controlled experiments delimit what can be inferred from this native-protocol comparison. With the initial layout controlled, PORTER and DSABRE are close on the reported main suite, while the layout source produces a larger EPR spread (Table III); the route budgets remain different, so this is not a budget-matched ranking of the routing loops. The ablations support a completion role for capacity relief and for the TELEGATE family under the tested capacity policy, but do not isolate a universal EPR benefit for either. On the six QFT and QPEexact scale-up instances, the fixed horizon portfolio accounts for the reported improvement, although its values were selected using the evaluation corpus. These observations do not attribute any individual native-protocol win to a single design choice: the compilers also differ in primitive vocabulary, candidate granularity, commitment, preparation scoring, layout construction, horizon policy, and route budget.

The primitive ablation identifies a comparator question, not a verdict against packets. On the eight 80-qubit instances completed by both full and no-packet variants, removing packets changes the geometric-mean EPR count by only 0.9%, and the direction varies by circuit. Disabling relief prevents completion on five of the nine 80-qubit instances, but the full configuration is 0.5% more expensive on the four instances completed by both relief variants. Strictly removing TELEGATE deadlocks seven of the nine; the reported teleport-only diagnostic also changes relief eligibility and is therefore not a one-factor ablation. Together, these results motivate a better-calibrated comparison between immediate gate resolution and the delayed benefit of a TELEDATA move. They do not establish that a restricted primitive set is globally preferable.

A. Future Work

Completion and progress guarantees. PORTER verifies every emitted route and uses implementation guards to terminate a run, but it does not claim successful completion for every feasible input. A useful next step is to separate the capacity and intra-core-progress questions. DSABRE's atomic relay mechanism may help establish a capacity argument, whereas the observed fixed-horizon failure on heavy-hex star Multiplier is a local-SWAP livelock with capacity still available. A completion proof for PORTER therefore also requires a progress argument for local preparation. Any transfer of a teleport-only guarantee to a richer primitive vocabulary should be proved or tested explicitly rather than inferred from the present ablation.

Horizon selection and scoring. All six scale-up instances select $L_{\mathrm{tp}} = 128$, whereas the configured one-route default is $L_{\mathrm{tp}} = 16$. This motivates two experiments. First, the fixed four-horizon portfolio should be validated on held-out circuit families or architectures. Second, a predictor should be tested for its ability to reduce the number of routing invocations without being fitted and evaluated on the same corpus. A per-qubit or horizon-limited interaction statistic is one candidate, because the scoring decision concerns moving one qubit over bounded future depth. The selection key also compares immediate CX resolution with predicted distance reduction. Calibrating these quantities on a common scale, or learning a constrained ranking rule using held-out data, would test whether the present heuristic comparator contributes to the vocabulary-dependent outcomes.

Scheduling and hardware-aware objectives. The output trace is unscheduled, so lower EPR consumption does not imply lower makespan or higher fidelity. A subsequent scheduler should exploit non-conflicting links, ports, and preparation SWAPs while accounting for entanglement generation rate, storage lifetime, and gate error [14], [15]. The resulting objective should be multi-objective: EPR pairs, depth, and fidelity can trade off differently across hardware. In particular, a TELEGATEPACKET keeps a cat resource live across a run; finite coherence or entanglement lifetime can limit useful packet length and may change the observed packet trade-off.

Interconnect and layout co-design. The evaluated machines provide one link between adjacent cores and one to four ports per core. Additional links or ports may create alternative preparations when one port is occupied, but they also alter capacity management and hardware cost. The heavy-hex star result further suggests that initial-layout reserve should depend on core-graph role rather than be uniform. A systematic study should vary link multiplicity, port placement, core-graph centrality, and reserve allocation, then report EPR, completion, and scheduling-aware objectives together.

VII. Related Work

This work is most closely related to online routers for multi-core distributed quantum computers that minimise EPR-pair consumption. It also relates to partition-first distributed compilation and to communication amortisation techniques.

Online EPR-aware routing. TeleSABRE [11] extends SABRE-style routing to support TeleData and TeleGate under an EPR-pair cost model. It scores executable local SWAP and inter-core actions together using a front/extended-set layout objective. Consequently, preparation for a remote operation is expressed through ordinary SWAPs and competes directly with immediately executable remote actions.

PORTER instead separates plan selection from plan preparation. It ranks a complete inter-core plan before generating plan-specific preparation SWAPs, commits to the selected plan, and then prepares it. It further adds same-control TeleGatePacket operations and capacity relief. These are algorithmic differences, not a claim that the two systems are evaluated under identical search budgets; Table I states the routing budget of every configuration reported here.

dSABRE [12] is the closest methodological comparison. Like PORTER, it is locality-first: it enters the inter-core stage only when no front CX can be handled within a core. Both treat free capacity as a legality condition, retain a per-core reserve whose threshold is two free positions in both defaults, and recover capacity by searching from a congested core to a reservoir. They differ in the available remote primitives, the decision unit, the role of preparation cost in scoring, and the layout protocol. dSABRE routes with TeleData; PORTER adds TeleGate and TeleGatePacket. dSABRE decides one core-hop at a time, toward a possibly more distant target core, and executes that hop atomically: the staging SWAPs it needs are performed inside the action, which is rolled back to a no-op if any step fails. PORTER decides a whole plan, including the primitive it will use, then prepares that plan with local SWAPs until it is ready and executes it, without reopening the remote choice. PORTER uses preparation cost as a tie-breaker after plan score, while dSABRE incorporates it in its move score. PORTER builds one deterministic initial layout, where dSABRE routes a best-of-N candidate set.

The two systems also differ in guarantees. Under its stated architectural assumptions, dSABRE safe mode provides a completion guarantee. PORTER guarantees correctness of emitted routes and implementation termination through guards, but does not claim completion for every feasible input (Section III-H). Completion in the reported horizon-portfolio experiments is empirical evidence, not a proof.

Partition-first and communication-amortising approaches. Partition-first compilers assign logical qubits to cores by hypergraph or community structure and route the remaining inter-core interactions [16]–[18]. The two-stage formulation dates to the earliest DQC compilers: one line divides the qubits among cores and then schedules teleportation-based migrations until every gate is local [19], another poses multi-core mapping as partitioning over time [20]. `pytket-dqc` [17] is used here as an external layout source, not as a directly comparable EPR-router baseline: its e-bit model assumes different entanglement lifetime and communication-memory conditions. AutoComm [9] identifies remote gate sequences that can share an entangled resource. PORTER adapts the amortisation idea inside its online routing loop through TeleGatePacket, rather than using a separate pre-pass. Finally, SABRE [13] supplies the front-plus-lookahead routing framework from which the online routers draw their scoring structure; within a single core that framework has been extended with deeper search [21] and with depth-aware objectives that account for single-qubit gates [22].

TABLE VIII
Design comparison. "Decision unit" is the object selected by the online routing decision. The guarantee row distinguishes a completion guarantee from implementation termination.

	TeleSABRE	dSABRE	`pytket-dqc`	PORTER
Paradigm	online	online	partition-first	online
Remote primitives	data, gate	data	n/a	data, gate, packet
Local/remote policy	joint pool	staged	n/a	staged
Decision unit	one action	one hop	n/a	whole plan
Preparation cost	in score	in score	n/a	tie-break
Layout protocol	Hungarian, 3 seeds	SabreLayout, 3 seeds	hypergraph partition	one deterministic
Capacity treatment	cap	legality + relay	n/a	legality + relief
Completion guarantee	not stated	conditional	n/a	not claimed
Primary metric	EPR pairs	EPR pairs	e-bits	EPR pairs

Table VIII summarises the comparison. "Not applicable" does not mean that a system lacks the capability; it means that the axis is not meaningfully comparable under its different compilation model.

VIII. Conclusion

PORTER is a deterministic distributed quantum circuit compiler whose inter-core decision unit is a complete plan. It ranks the progress that a remote action can deliver before generating plan-specific preparation SWAPs, commits to the selected plan, and then prepares it. The implementation combines this plan-oriented stage with capacity relief, a deterministic initial layout, and a teleport-lookahead horizon L_{tp} chosen per circuit from a fixed four-value portfolio.

Under the reported native protocols, PORTER uses substantially fewer EPR pairs than TeleSABRE and completes more instances, and it is competitive with dSABRE rather than uniformly superior: the two obtain their lower counts on different instances, and backward refinement moves PORTER ahead on the pooled mean at a compile-time cost. The controlled experiments locate the sensitivities. Capacity relief and gate teleportation are needed for completion rather than for cost; the choice of initial layout affects EPR more than the choice between the two routing loops; and the lookahead horizon accounts for the scale-up outcome. Establishing completion and local-progress guarantees, validating horizon selection out of sample, calibrating the plan comparator, and evaluating scheduling, fidelity, and interconnect trade-offs under hardware-aware cost models remain open (Section VI-A).

Acknowledgments

This work was partially supported by the Natural Science Foundation of China (Grant No. 12471437). The authors used

Anthropic's Claude (via Claude Code) to assist with the PORTER implementation, the replay verifier and comparison harness, and the drafting of this manuscript. All AI-generated content was reviewed and verified by the authors, who take full responsibility for the correctness and originality of the work.

References

[1] D. Cuomo, M. Caleffi, and A. S. Cacciapuoti, "Towards a distributed quantum computing ecosystem," *IET Quantum Communication*, vol. 1, no. 1, pp. 3–8, 2020.

[2] P. Escofet, A. Ovide, M. Bandic, L. Prielinger, H. Van Someren, S. Feld, E. Alarcón, S. Abadal, and C. G. Almudever, "Revisiting the mapping of quantum circuits: Entering the multi-core era," *ACM Transactions on Quantum Computing*, vol. 6, no. 1, pp. 4:1–4:26, 2025.

[3] J. Ang, G. Carini, Y. Chen, I. Chuang, M. DeMarco *et al.*, "ARQUIN: Architectures for multinode superconducting quantum computers," *ACM Transactions on Quantum Computing*, vol. 5, no. 3, pp. 19:1–19:59, 2024.

[4] L. J. Stephenson, D. P. Nadlinger, B. C. Nichol, S. An, P. Drmota, T. G. Ballance, K. Thirumalai, J. F. Goodwin, D. M. Lucas, and C. J. Ballance, "High-rate, high-fidelity entanglement of qubits across an elementary quantum network," *Physical Review Letters*, vol. 124, no. 11, p. 110501, 2020.

[5] S. Daiss, S. Langenfeld, S. Welte, E. Distante, P. Thomas, L. Hartung, O. Morin, and G. Rempe, "A quantum-logic gate between distant quantum-network modules," *Science*, vol. 371, no. 6529, pp. 614–617, 2021.

[6] C. H. Bennett, G. Brassard, C. Crépeau, R. Jozsa, A. Peres, and W. K. Wootters, "Teleporting an unknown quantum state via dual classical and Einstein-Podolsky-Rosen channels," *Physical Review Letters*, vol. 70, no. 13, pp. 1895–1899, 1993.

[7] J. Eisert, K. Jacobs, P. Papadopoulos, and M. B. Plenio, "Optimal local implementation of nonlocal quantum gates," *Physical Review A*, vol. 62, no. 5, p. 052317, 2000.

[8] A. Yimsiriwattana and S. J. Lomonaco Jr., "Generalized GHZ states and distributed quantum computing," *arXiv preprint quant-ph/0402148*, 2004.

[9] A. Wu, H. Zhang, G. Li, A. Shabani, Y. Xie, and Y. Ding, "AutoComm: A framework for enabling efficient communication in distributed quantum programs," in *Proceedings of the 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022, pp. 1027–1041, arXiv:2207.11674.

[10] N. Quetschlich, L. Burgholzer, and R. Wille, "MQT Bench: Benchmarking software and design automation tools for quantum computing," *Quantum*, vol. 7, p. 1062, 2023.

[11] E. Russo, E. Vinciguerra, M. Palesi, D. Patti, G. Ascia, and V. Catania, "TeleSABRE: Heuristic layout synthesis in multi-core quantum systems with teleport interconnect," in *Proceedings of the IEEE International Conference on Quantum Computing and Engineering (QCE)*, 2025, pp. 749–758, arXiv:2505.08928. Code: https://github.com/Haimrich/telesabre.

[12] S. Li, "dSABRE: A SABRE-style router for multi-core distributed quantum computers," 2026, manuscript, Centre for Quantum Software and Information, University of Technology Sydney. Code: https://github.com/ebony72/dsabre.

[13] G. Li, Y. Ding, and Y. Xie, "Tackling the qubit mapping problem for NISQ-era quantum devices," in *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2019, pp. 1001–1014.

[14] J. M. Baker, C. Duckering, A. Hoover, and F. T. Chong, "Time-sliced quantum circuit partitioning for modular architectures," in *Proceedings of the 17th ACM International Conference on Computing Frontiers (CF)*, 2020, pp. 98–107.

[15] D. Ferrari, A. S. Cacciapuoti, M. Amoretti, and M. Caleffi, "Compiler design for distributed quantum computing," *IEEE Transactions on Quantum Engineering*, vol. 2, pp. 1–20, 2021.

[16] P. Escofet, A. Ovide, C. G. Almudever, E. Alarcón, and S. Abadal, "Hungarian qubit assignment for optimized mapping of quantum circuits on multi-core architectures," *IEEE Computer Architecture Letters*, vol. 22, no. 2, pp. 161–164, 2023.

[17] P. Andres-Martinez, D. Mills, T. Forrer, and L. Henaut, "CQCL/pytket-dqc," https://github.com/CQCL/pytket-dqc, Jun. 2024.

[18] O. Daei, K. Navi, and M. Zomorodi-Moghadam, "Optimized quantum circuit partitioning," *International Journal of Theoretical Physics*, vol. 59, pp. 3804–3820, 2020.

[19] R. G. Sundaram, H. Gupta, and C. R. Ramakrishnan, "Efficient distribution of quantum circuits," in *Proceedings of the 35th International Symposium on Distributed Computing (DISC)*, ser. LIPIcs, vol. 209, 2021, pp. 41:1–41:20.

[20] A. Ovide, S. Rodrigo, M. Bandic, H. Van Someren, S. Feld, S. Abadal, E. Alarcón, and C. G. Almudever, "Mapping quantum algorithms to multi-core quantum computing architectures," in *Proceedings of the IEEE International Symposium on Circuits and Systems (ISCAS)*, 2023, pp. 1–5, arXiv:2303.16125.

[21] X. Zhou, Y. Feng, and S. Li, "Quantum circuit transformation: A Monte Carlo tree search framework," *ACM Transactions on Design Automation of Electronic Systems*, vol. 27, no. 6, pp. 59:1–59:27, 2022.

[22] S. Li, K. D. Nguyen, Z. Clare, and Y. Feng, "Single-qubit gates matter for optimising quantum circuit depth in qubit mapping," in *Proceedings of the IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, 2023.



Sanjiang Li received his B.Sc. in mathematics from Shaanxi Normal University in 1996 and his Ph.D. in mathematics from Sichuan University in 2001. He is currently a professor at the Centre for Quantum Software and Information at the University of Technology Sydney (UTS). Prior to joining UTS, he worked in the Department of Computer Science and Technology at Tsinghua University from 2001 to 2008. His primary research interests include knowledge representation, artificial intelligence, and quantum circuit compilation.